

Reporte Ejecutivo: Afinidad de Canasta para TCA

Diego Armando Mijares Ledezma (A01722421)
Daniel Eduardo Arana Bodart (A01741202)
Mauricio Octavio Valencia Gonzalez (A01234397)
David Alejandro Acuña Orozco (A00571187)
Diego Gutiérrez Vargas (A01285421)
Alfonso Elizondo Partida (A01285151)

7 de junio de 2026

Resumen Ejecutivo

TCA, junto a su cliente Merksyst identificó una oportunidad de cross-selling en su ERP: clientes con 5.19 artículos/ticket y ~30,000 transacciones mensuales por tienda. El caso de uso ANA-03 desarrolló un motor de recomendación Early-Fusion Transformer (RNN + TDA + Series de Tiempo) entrenado sobre 688,322 transacciones reales (7,792 SKUs), que sugiere los 10 productos más probables a agregar a un pedido. Con un hit rate del 31 % (proxy *offline* a validar en producción), proyecta un beneficio de \$361,584 MXN/mes, VPN de \$5,497,433 MXN, TIR de 64.1 % y payback de 20 meses.

1. Introducción

TCA Software Solutions provee ERP para el sector retail de abarrotes y conveniencia en México; el 50 % de sus clientes con más de dos formatos de venta los desarrollaron tras su adopción. Aunque sus tickets ya superan el benchmark del sector (5.19 vs. 3.5–4.0 artículos), el alto volumen de 30,000 transacciones/mes amplifica el impacto de un sistema de recomendación proactiva. El vendedor hoy solo atiende lo que el cliente solicita explícitamente, sin herramientas para sugerir complementos basados en patrones históricos.

Para resolver esto se tomó el caso de uso ANA-03: un motor basado en Early-Fusion Transformer desplegado como API REST en Azure, integrable con el ERP sin modificar la infraestructura existente. El presente reporte resume los resultados, impacto financiero y recomendaciones estratégicas para su adopción.

2. Objetivos Generales

- Incrementar el ticket promedio mediante recomendaciones automatizadas, reduciendo la venta reactiva.
- Desarrollar un modelo Early-Fusion Transformer que identifique patrones de co-compra entre los ~3,000 SKUs activos.
- Implementar un servicio containerizado en la nube integrable con el ERP utilizado por Merksyst sin cambios en la infraestructura.
- Asegurar equidad operativa en la exposición de productos: cobertura de catálogo y diversidad, evitando sobre-recomendar únicamente los artículos más populares.
- Sentar bases técnicas para mantenimiento, reentrenamiento y escalabilidad del sistema.

3. Metodología

El proyecto se desarrolló mediante una metodología SCRUM. Se realizaron dos sprints principales: el primero abarcó la exploración, limpieza de datos, selección de arquitectura, entrenamiento y optimización del modelo con Optuna, además del pipeline de orquestación con Kedro; el segundo sprint contempló el despliegue en Azure, pruebas de la API y el desarrollo del front-end en HTML para las interfaces de usuario. Este enfoque permitió validar avances de manera periódica y asegurar que la solución respondiera a las necesidades reales del proceso comercial.

3.1. Comprensión del negocio y gobierno de datos

En una primera etapa se analizó el proceso de venta de Merksyst y la información disponible dentro del ERP, incluyendo transacciones, catálogo de productos e inventarios. A partir de este análisis se utilizó el caso de uso ANA-03 y se establecieron lineamientos para el manejo responsable de la información, garantizando la confidencialidad, trazabilidad y calidad de los datos utilizados durante el proyecto.

3.2. Preparación y calidad de los datos

Mediante un *pipeline* de limpieza, se integraron las fuentes de información y se eliminaron anomalías como cancelaciones, registros sin clave, cantidades no positivas y errores de formato. Para garantizar la consistencia, se filtraron transacciones con presencia ≥ 4 semanas y productos con más de 20 ventas en ≥ 20 *tickets* distintos. La base final consolidó 688,322 transacciones y 7,792 SKUs únicos, utilizando Python, DuckDB/SQL sobre archivos *.parquet*, y las librerías *Pandas*, *NumPy*, *Matplotlib* y *Statsmodels*.

3.3. Desarrollo del modelo de recomendación

Con la información preparada, se desarrolló el modelo Early-Fusion Transformer, que integra tres señales complementarias, generadas por módulos especializados, mediante una capa de fusión basada en atención:

RNN: captura recurrencias y frecuencias entre productos en canastas históricas. **TDA (MAPPER):** analiza la estructura topológica de los datos, identificando relaciones entre clusters de productos no evidentes en el espacio euclidiano. **Series de Tiempo:** pondera la tendencia reciente de cada SKU mediante pruebas ADF (estacionariedad) y análisis de autocorrelación por rezagos. **Transformer (Early-Fusion):** capa de fusión que integra los scores de los tres módulos anteriores para generar las 10 recomendaciones finales dado un conjunto de productos en el carrito.

Los hiperparámetros fueron optimizados automáticamente con Optuna (muestreador TPE + MedianPruner), maximizando el Mean Reciprocal Rank (MRR) en validación. La métrica Hit@10 mide la proporción de casos en que el producto que el cliente realmente agregó aparece dentro de las 10 recomendaciones sugeridas.

3.4. Implementación tecnológica

El sistema fue empaquetado con Docker y desplegado como servicio en Microsoft Azure Container Apps, expuesto mediante una API REST con FastAPI a través del endpoint `POST /recommend`. Esta arquitectura permite que el ERP consulte recomendaciones en tiempo real sin requerir modificaciones en la infraestructura tecnológica existente. Se desarrollaron dos interfaces de usuario: una vista orientada al cliente y una vista para el vendedor que muestra las recomendaciones con scores comparativos en tiempo de escaneo.

3.5. Validación, enfoque ético y documentación

Se definieron criterios de validación para evaluar la consistencia y utilidad de las recomendaciones generadas. Se incorporó un Modelo de Justicia con seis tipos de sesgo identificados. Dado que los datos transaccionales no contienen identificadores ni atributos demográficos de clientes, la equidad se evalúa en esta etapa a nivel operativo y de exposición de productos: cobertura de catálogo, diversidad por familia y disponibilidad de inventario. Las métricas demográficas (paridad, igualdad de oportunidades e impacto dispar, con umbral de alerta $< 0,8$) se incorporarían en una versión futura que utilice identificadores de cliente. Se generó documentación completa incluyendo guía de integración para desarrolladores y repositorio de código estructurado.

4. Principales hallazgos y Conclusiones

4.1. KPIs y métricas de impacto

Tabla 1: Indicadores clave de desempeño: ANA-03

KPI	¿Qué mide?	Actual	12m	24m	Bench. MX
Artículos/ticket	Promedio de productos por transacción	5.19	6.0	6.5	3.5–4.0
Ticket promedio	Valor monetario por transacción	\$249	\$292	\$320	\$350–\$450
Tasa adopción motor	Uso de recomendaciones por vendedor	0 %	35 %	60 %	40–65 %
Conversión cross-sell	% sugerencias → compra	N/A	15 %	31 %	15–30 %
Hit@10 del modelo	Producto real dentro del top-10 sugerido	—	31 %	optimizado	—
Cobertura catálogo	% SKUs con recomendaciones	0 %	55 %	75 %	—
Disponibilidad API	SLA del servicio	—	99 %	99.5 %	99 %+

4.2. Gobierno de Datos y Modelo de Justicia

El gobierno de datos garantiza cinco pilares: **calidad** (estrategias de limpieza sobre datos ERP), **confidencialidad** (datos transaccionales nunca versionados públicamente), **trazabilidad** (ruta documentada desde ERP hasta recomendación), **equidad operativa** (seis tipos de sesgo identificados; métricas de cobertura de catálogo, diversidad por familia y disponibilidad operativa) y **roles** definidos (Data Steward, Data Engineer, Data Scientist, MLOps, Responsable de ética).

Los seis sesgos identificados y sus mitigaciones son: **histórico** (auditoría de la distribución de transacciones por producto, categoría y sucursal antes del entrenamiento), **reporte** (monitoreo de aceptación en producción), **popularidad / cold-start de ítem** (componente de exploración para exponer productos nuevos o de baja rotación), **confirmación** (selección objetiva por MRR y umbrales documentados), **retroalimentación** (exploración para diversidad controlada) y **disponibilidad operativa** (integración de inventario como restricción de negocio).

4.3. Retorno de Inversión y viabilidad financiera

El beneficio se calcula sobre la palanca de cross-selling: $30,000 \times 31 \% \times 0,81 \times \$48 = \$361,584$ MXN/mes $\Rightarrow$ \$4,339,008 MXN/año en operación plena.

Tabla 2: Indicadores financieros y flujo de efectivo proyectado

Concepto	Indicador	Año 0	Año 1 (40 %)	Año 2 (75 %)	Año 3 (100 %)	Año 4 (100 %)
VPN	\$5,497,433 MXN	3*(\$3,693,000)				
TIR	64.1 %					
Payback	20 meses					
(+) Beneficios		—	\$1,735,603	\$3,254,256	\$4,339,008	\$4,339,008
(-) FTE + Azure + mant.	WACC 14 %	(\$3,693,000)	(\$114,568)	(\$114,568)	(\$114,568)	(\$114,568)
Flujo neto		(\$3,693,000)	\$1,621,035	\$3,139,688	\$4,224,440	\$4,224,440
Flujo acumulado		(\$3,693,000)	(\$2,071,965)	\$1,067,723	\$5,292,163	\$9,516,603

Gastos operativos = Azure \$22,243 MXN/año (margen +85 %) + FTE mantenimiento 2.5 % (\$92,325 MXN/año). Inv. base ponderada académica: \$4,747,840 MXN.

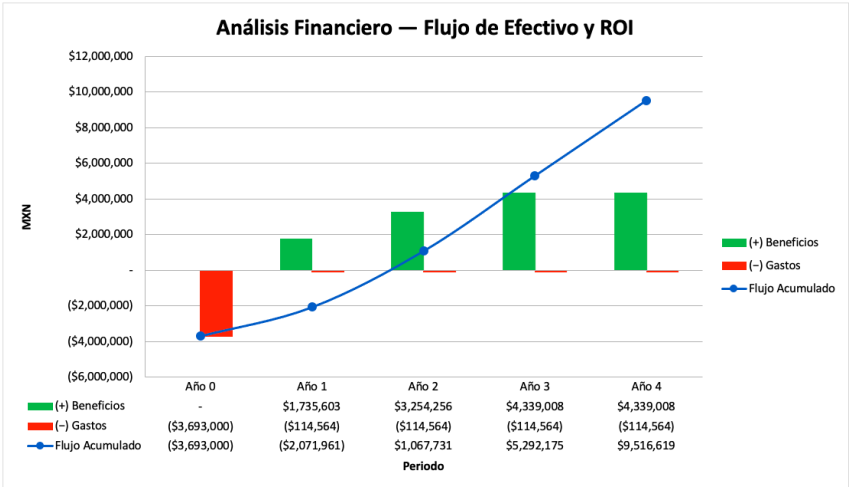


Figura 1: Flujo de efectivo anual: Beneficios (verde) vs. Costos (rojo) y flujo acumulado

4.4. Salarios del equipo y costo de implementación

Desglose de roles requeridos para el desarrollo e implementación del sistema, con referencia a los salarios promedio del mercado laboral mexicano:

Tabla 3: Costo estimado del equipo de implementación

Rol	Cantidad	Salario mínimo	Salario máximo	Costo total
Data Scientist (IDM)	6	\$35,000	\$55,000	\$2,520,000
Data Engineer	1	\$38,000	\$60,000	\$228,000
Database Administrator	1	\$35,000	\$50,000	\$105,000
MLOps Engineer	1	\$45,000	\$70,000	\$360,000
Project Manager	1	\$40,000	\$65,000	\$480,000
TOTAL FTE	10	—	—	\$3,693,000

4.5. Interfaz y ejemplo de uso

El sistema ofrece dos vistas: **vista cliente** (estilo carrito, recomendaciones visuales en tiempo real) y **vista vendedor/cajero** (top-10 con scores mientras escanea artículos). Un hallazgo

cualitativo destacado: en canastas con chiles jalapeños, el modelo recomienda pan, catsup, mayonesa, cebolla, tocino y aguacate, inferiendo el contexto de preparación completo; no solo artículos similares.

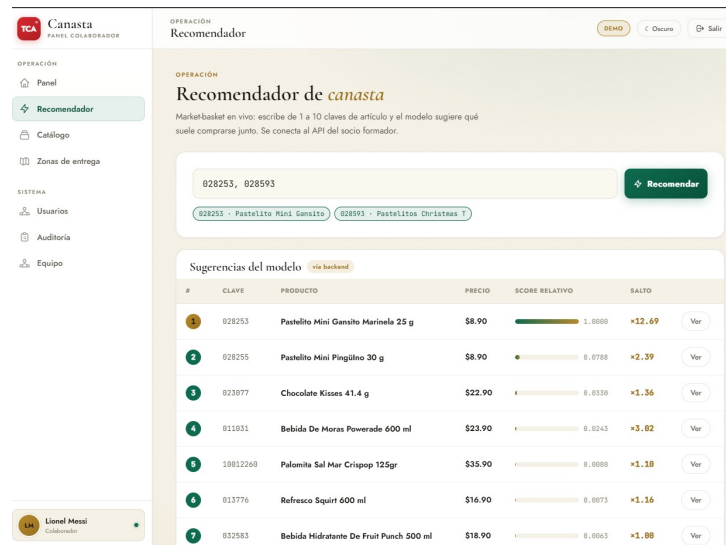


Figura 2: Interfaz del recomendador de canasta: vista de resultados

5. Recomendaciones

1. **Integrar POST /recommend en el flujo de venta del ERP.** Sin modificar la arquitectura existente, el vendedor visualiza el top-10 en tiempo real durante la captura del pedido.
2. **Reentrenar cada 3–6 meses.** Los patrones de compra evolucionan con temporadas y nuevos productos. Costo estimado: \$613 MXN/año (2 ejecuciones en Azure CPU).
3. **Activar instancia permanente en Azure.** Elimina el cold start de 10–20 s en producción con múltiples cajeros (\$35–50 USD/mes adicionales).
4. **Monitorear equidad operativa en producción.** Vigilar cobertura de catálogo, concentración de exposición y disponibilidad de inventario; reservar el ratio de impacto dispar ($< 0,8$) para una futura versión con identificadores de cliente.
5. **Expandir hacia combos y campañas.** Usar los rankings del motor para crear bundles en catálogo y campañas dirigidas con artículo ancla.
6. **Escalar el vocabulario.** Si el catálogo supera los $\sim 3,000$ SKUs actuales, ampliar el vocabulario y reentrenar. A futuro: compatibilidad con procesos TCA e imágenes de productos en la interfaz.

6. Conclusión

A partir de 688,322 transacciones reales, el modelo aprendió patrones de co-compra no evidentes y los sirve en tiempo real con un hit rate del 31 % (proxy *offline*). La inversión inicial de \$3,693,000 MXN se recupera en 20 meses, con un VPN de \$5,497,433 MXN y TIR de 64.1 %, ambos muy por encima del WACC del 14 %. Este rendimiento se sustenta en el beneficio anual proyectado para Merksyst de \$4,339,008 MXN por tienda en operación plena, demostrando el alto valor agregado que TCA puede integrar a su ERP. Asimismo, el proyecto sienta las bases para casos de uso adicionales: combos, campañas y optimización de inventario por afinidad de canasta.